4.5	2.0
5.5	3.5
6.5	5.0
7.5	6.0
8.5	7.5
8.0	9.0
9.0	10.5
9.5	12.0
10.0	14.0

(s) The **X** and **Y** coordinates of 10 different points are entered through the keyboard. Write a program to find the distance of last point from the first point (sum of distance between consecutive points).

9 Puppetting On Strings

- What are Strings
- More about Strings
- Pointers and Strings
- Standard Library String Functions
  - *strlen( )*
  - *strcpy*( )
  - *strcat*( )
  - *strcmp*( )
- Two-Dimensional Array of Characters
- Array of Pointers to Strings
- Limitation of Array of Pointers to Strings
  - Solution
- Summary
- Exercise

In the last chapter you learnt how to define arrays of differing sizes and dimensions, how to initialize arrays, how to pass arrays to a function, etc. With this knowledge under your belt, you should be ready to handle strings, which are, simply put, a special kind of array. And strings, the ways to manipulate them, and how pointers are related to strings are going to be the topics of discussion in this chapter.

What are Strings

The way a group of integers can be stored in an integer array, similarly a group of characters can be stored in a character array. Character arrays are many a time also called strings. Many languages internally treat strings as character arrays, but somehow conceal this fact from the programmer. Character arrays or strings are used by programming languages to manipulate text such as words and sentences.

A string constant is a one-dimensional array of characters terminated by a null ('\0'). For example,

```
char name[ ] = { 'H', 'A', 'E', 'S', 'L', 'E', 'R', '\0' } ;
```

Each character in the array occupies one byte of memory and the last character is always '\0'. What character is this? It looks like two characters, but it is actually only one character, with the \ indicating that what follows it is something special. '\0' is called null character. Note that '\0' and '0' are not same. ASCII value of '\0' is 0, whereas ASCII value of '0' is 48. Figure 9.1 shows the way a character array is stored in memory. Note that the elements of the character array are stored in contiguous memory locations.

The terminating null ('\0') is important, because it is the only way the functions that work with a string can know where the string ends. In fact, a string not terminated by a '\0' is not really a string, but merely a collection of characters.

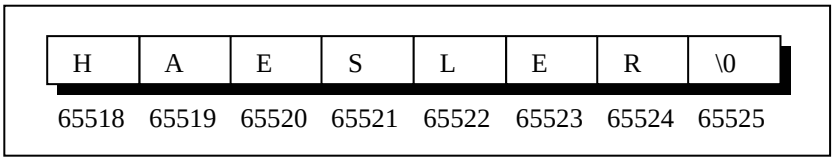

Figure 9.1

C concedes the fact that you would use strings very often and hence provides a shortcut for initializing strings. For example, the string used above can also be initialized as,

```
char  name[ ] = "HAESLER" ;
```

Note that, in this declaration '\0' is not necessary. C inserts the null character automatically.

More about Strings

In what way are character arrays different than numeric arrays? Can elements in a character array be accessed in the same way as the elements of a numeric array? Do I need to take any special care of '\0'? Why numeric arrays don't end with a '\0'? Declaring strings is okay, but how do I manipulate them? Questions galore!! Well, let us settle some of these issues right away with the help of sample programs.

```
/* Program to demonstrate printing of a string */
main( )
{
      char  name[ ] = "Klinsman" ;
      int  i = 0 ;

      while ( i <= 7 )
      {
            printf ( "%c", name[i] ) ;
            i++ ;
      }
```

```
}
```

And here is the output...

```
Klinsman
```

No big deal. We have initialized a character array, and then printed out the elements of this array within a **while** loop. Can we write the **while** loop without using the final value 7? We can; because we know that each character array always ends with a '\0'. Following program illustrates this.

```
main( )
{
    char  name[ ] = "Klinsman" ;
    int  i = 0 ;

    while ( name[i] != `\0' )
    {
        printf ( "%c", name[i] ) ;
        i++ ;
    }
}
```

And here is the output...

```
Klinsman
```

This program doesn't rely on the length of the string (number of characters in it) to print out its contents and hence is definitely more general than the earlier one. Here is another version of the same program; this one uses a pointer to access the array elements.

```
main( )
{
    char  name[ ] = "Klinsman" ;
    char  *ptr ;
```

```
    ptr = name ;  /* store base address of string */

    while ( *ptr != `\0' )
    {
        printf ( "%c", *ptr ) ;
        ptr++ ;
    }
}
```

As with the integer array, by mentioning the name of the array we get the base address (address of the zeroth element) of the array. This base address is stored in the variable **ptr** using,

```
ptr = name ;
```

Once the base address is obtained in **ptr**, ***ptr** would yield the value at this address, which gets printed promptly through,

```
printf ( "%c", *ptr ) ;
```

Then, **ptr** is incremented to point to the next character in the string. This derives from two facts: array elements are stored in contiguous memory locations and on incrementing a pointer it points to the immediately next location of its type. This process is carried out till **ptr** doesn't point to the last character in the string, that is, '\0'.

In fact, the character array elements can be accessed exactly in the same way as the elements of an integer array. Thus, all the following notations refer to the same element:

```
name[i]
*( name + i )
*( i + name )
i[name]
```

Even though there are so many ways (as shown above) to refer to the elements of a character array, rarely is any one of them used. This is because **printf( )** function has got a sweet and simple way of doing it, as shown below. Note that **printf( )** doesn't print the '\0'.

```
main( )
{
    char  name[ ] = "Klinsman" ;
    printf ( "%s", name ) ;
}
```

The **%s** used in **printf( )** is a format specification for printing out a string. The same specification can be used to receive a string from the keyboard, as shown below.

```
main( )
{
    char  name[25] ;

    printf ( "Enter your name " ) ;
    scanf ( "%s", name ) ;
    printf ( "Hello %s!", name ) ;
}
```

And here is a sample run of the program...

```
Enter your name Debashish
Hello Debashish!
```

Note that the declaration **char name[25]** sets aside 25 bytes under the array **name[ ],** whereas the **scanf( )** function fills in the characters typed at keyboard into this array until the enter key is hit. Once enter is hit, **scanf( )** places a '\0' in the array. Naturally, we should pass the base address of the array to the **scanf( )** function.

While entering the string using **scanf( )** we must be cautious about two things:

(a) The length of the string should not exceed the dimension of the character array. This is because the C compiler doesn't perform bounds checking on character arrays. Hence, if you carelessly exceed the bounds there is always a danger of overwriting something important, and in that event, you would have nobody to blame but yourselves.

(b) **scanf( )** is not capable of receiving multi-word strings. Therefore names such as 'Debashish Roy' would be unacceptable. The way to get around this limitation is by using the function **gets( )**. The usage of functions **gets( )** and its counterpart **puts( )** is shown below.

```
main( )
{
    char  name[25] ;

    printf ( "Enter your full name " ) ;
    gets ( name ) ;
    puts ( "Hello!" ) ;
    puts ( name ) ;
}
```

And here is the output...

```
Enter your name Debashish Roy
Hello!
Debashish Roy
```

The program and the output are self-explanatory except for the fact that, **puts( )** can display only one string at a time (hence the use of two **puts( )** in the program above). Also, on displaying a string, unlike **printf( )**, **puts( )** places the cursor on the next line. Though **gets( )** is capable of receiving only

one string at a time, the plus point with **gets( )** is that it can receive a multi-word string.

If we are prepared to take the trouble we can make **scanf( )** accept multi-word strings by writing it in this manner:

```
char  name[25] ;
printf ( "Enter your full name " ) ;
scanf ( "%[^\n]s", name ) ;
```

Though workable this is the best of the ways to call a function, you would agree.

Pointers and Strings

Suppose we wish to store “Hello”. We may either store it in a string or we may ask the C compiler to store it at some location in memory and assign the address of the string in a **char** pointer. This is shown below:

```
char  str[ ] = "Hello" ;
char  *p = "Hello" ;
```

There is a subtle difference in usage of these two forms. For example, we cannot assign a string to another, whereas, we can assign a **char** pointer to another **char** pointer. This is shown in the following program.

```
main( )
{
    char  str1[ ] = "Hello" ;
    char  str2[10] ;

    char  *s = "Good Morning" ;
    char  *q ;
```

```
    str2 = str1 ; /* error */
    q = s ; /* works */
}
```

Also, once a string has been defined it cannot be initialized to another set of characters. Unlike strings, such an operation is perfectly valid with **char** pointers.

```
main( )
{
    char  str1[ ] = "Hello" ;
    char  *p = "Hello" ;
    str1 = "Bye" ; /* error */
    p = "Bye" ; /* works */
}
```

Standard Library String Functions

With every C compiler a large set of useful string handling library functions are provided. Figure 9.2 lists the more commonly used functions along with their purpose.

Function	Use
strlen	Finds length of a string
strlwr	Converts a string to lowercase
strupr	Converts a string to uppercase
strcat	Appends one string at the end of another
strncat	Appends first n characters of a string at the end of another

strcpy	Copies a string into another
strncpy	Copies first n characters of one string into another
strcmp	Compares two strings
strncmp	Compares first n characters of two strings
strcmpi	Compares two strings without regard to case ("i" denotes that this function ignores case)
stricmp	Compares two strings without regard to case (identical to strcmpi)
strnicmp	Compares first n characters of two strings without regard to case
strdup	Duplicates a string
strchr	Finds first occurrence of a given character in a string
strrchr	Finds last occurrence of a given character in a string
strstr	Finds first occurrence of a given string in another string
strset	Sets all characters of string to a given character
strnset	Sets first n characters of a string to a given character
strrev	Reverses string

Figure 9.2

Out of the above list we shall discuss the functions **strlen( )**, **strcpy( )**, **strcat( )** and **strcmp( )**, since these are the most commonly used functions. This will also illustrate how the library functions in general handle strings. Let us study these functions one by one.

strlen( )

This function counts the number of characters present in a string. Its usage is illustrated in the following program.

```
main( )
{
    char  arr[ ] = "Bamboozled" ;
    int  len1, len2 ;
```

```
    len1 = strlen ( arr ) ;
    len2 = strlen ( "Humpty Dumpty" ) ;

    printf ( "\nstring = %s length = %d", arr, len1 ) ;
    printf ( "\nstring = %s length = %d", "Humpty Dumpty", len2 ) ;
}
```

The output would be...

```
string = Bamboozled length = 10
string = Humpty Dumpty length = 13
```

Note that in the first call to the function **strlen( )**, we are passing the base address of the string, and the function in turn returns the length of the string. While calculating the length it doesn't count '\0'. Even in the second call,

```
len2 = strlen ( "Humpty Dumpty" ) ;
```

what gets passed to **strlen( )** is the address of the string and not the string itself. Can we not write a function **xstrlen( )** which imitates the standard library function **strlen( )**? Let us give it a try...

```
/* A look-alike of the function strlen( ) */
main( )
{
    char  arr[ ] = "Bamboozled" ;
    int  len1, len2 ;

    len1 = xstrlen ( arr ) ;
    len2 = xstrlen ( "Humpty Dumpty" ) ;

    printf ( "\nstring = %s length = %d", arr, len1 ) ;
    printf ( "\nstring = %s length = %d", "Humpty Dumpty", len2 ) ;
}
```

```
xstrlen ( char  *s )
{
      int  length = 0 ;

      while ( *s != '\0' )
      {
            length++ ;
            s++ ;
      }

      return ( length ) ;
}
```

The output would be...

```
string = Bamboozled length = 10
string = Humpty Dumpty length = 13
```

The function **xstrlen( )** is fairly simple. All that it does is keep counting the characters till the end of string is not met. Or in other words keep counting characters till the pointer s doesn't point to '\0'.

strcpy( )

This function copies the contents of one string into another. The base addresses of the source and target strings should be supplied to this function. Here is an example of **strcpy( )** in action...

```
main( )
{
      char  source[ ] = "Sayonara" ;
```

```
    char  target[20] ;

    strcpy ( target, source ) ;
    printf ( "\nsource string = %s", source ) ;
    printf ( "\ntarget string = %s", target ) ;
}
```

And here is the output...

```
source string = Sayonara
target string = Sayonara
```

On supplying the base addresses, **strcpy( )** goes on copying the characters in source string into the target string till it doesn't encounter the end of source string (‘\0’). It is our responsibility to see to it that the target string’s dimension is big enough to hold the string being copied into it. Thus, a string gets copied into another, piece-meal, character by character. There is no short cut for this. Let us now attempt to mimic **strcpy( )**, via our own string copy function, which we will call **xstrcpy( ).**

```
main( )
{
    char  source[ ] = "Sayonara" ;
    char  target[20] ;

    xstrcpy ( target, source ) ;
    printf ( "\nsource string = %s", source ) ;
    printf ( "\ntarget string = %s", target ) ;
}

xstrcpy ( char  *t, char  *s )
{
    while ( *s != '\0' )
    {
        *t = *s ;
        s++ ;
```

```
            t++ ;
        }
        *t = '\0' ;
}
```

The output of the program would be...

```
source string = Sayonara
target string = Sayonara
```

Note that having copied the entire source string into the target string, it is necessary to place a '\0' into the target string, to mark its end.

If you look at the prototype of **strcpy( )** standard library function, it looks like this…

```
strcpy ( char *t, const char *s ) ;
```

We didn't use the keyword **const** in our version of **xstrcpy( )** and still our function worked correctly. So what is the need of the **const** qualifier?

What would happen if we add the following lines beyond the last statement of **xstrcpy( )**?.

```
s = s - 8 ;
*s = 'K' ;
```

This would change the source string to "Kayonara". Can we not ensure that the source string doesn't change even accidentally in **xstrcpy( )**? We can, by changing the definition as follows:

```
void xstrcpy ( char *t, const char *s )
{
    while ( *s != '\0' )
    {
```

```
            *t = *s ;
            s++ ;
            t++ ;
        }
        *t = '\0' ;
}
```

By declaring **char *s** as **const** we are declaring that the source string should remain constant (should not change). Thus the **const** qualifier ensures that your program does not inadvertently alter a variable that you intended to be a constant. It also reminds anybody reading the program listing that the variable is not intended to change.

We can use **const** in several situations. The following code fragment would help you to fix your ideas about **const** further.

```
char *p = "Hello" ;  /* pointer is variable, so is string */
*p = 'M' ;  /* works */
p = "Bye" ;  /* works */

const char *q = "Hello" ;  /* string is fixed pointer is not */
*q = 'M' ;  /* error */
q = "Bye" ;  /* works */

char const *s = "Hello" ;  /* string is fixed pointer is not */
*s = 'M' ;  /* error */
s = "Bye" ;  /* works */

char * const t = "Hello" ;  /* pointer is fixed string is not */
*t = 'M' ;  /* works */
t = "Bye" ;  /* error */

const char * const u = "Hello" ;  /* string is fixed so is pointer */
*u = 'M' ;  /* error */
u = "Bye" ;  /* error */
```

The keyword **const** can be used in context of ordinary variables like **int**, **float**, etc. The following program shows how this can be done.

```
main( )
{
    float r, a ;
    const float pi = 3.14 ;

    printf ( "\nEnter radius of circle " ) ;
    scanf  ( "%f", &r ) ;
    a = pi * r * r ;
    printf ( "\nArea of circle = %f", a ) ;
}
```

strcat( )

This function concatenates the source string at the end of the target string. For example, “Bombay” and “Nagpur” on concatenation would result into a string “BombayNagpur”. Here is an example of **strcat( )** at work.

```
main( )
{
    char  source[ ] = "Folks!" ;
    char  target[30] = "Hello" ;

    strcat ( target, source ) ;
    printf ( "\nsource string = %s", source ) ;
    printf ( "\ntarget string = %s", target ) ;
}
```

And here is the output...

```
source string = Folks!
target string = HelloFolks!
```

Note that the target string has been made big enough to hold the final string. I leave it to you to develop your own **xstrcat( )** on lines of **xstrlen( )** and **xstrcpy( ).**

strcmp( )

This is a function which compares two strings to find out whether they are same or different. The two strings are compared character by character until there is a mismatch or end of one of the strings is reached, whichever occurs first. If the two strings are identical, **strcmp( )** returns a value zero. If they're not, it returns the numeric difference between the ASCII values of the first non-matching pairs of characters. Here is a program which puts **strcmp( )** in action.

```
main( )
{
    char  string1[ ] = "Jerry" ;
    char  string2[ ] = "Ferry" ;
    int  i, j, k ;

    i = strcmp ( string1, "Jerry" ) ;
    j = strcmp ( string1, string2 ) ;
    k = strcmp ( string1, "Jerry boy" ) ;

    printf ( "\n%d %d %d", i, j, k ) ;
}
```

And here is the output...

```
0 4 -32
```

In the first call to **strcmp( )**, the two strings are identical—"Jerry" and "Jerry"—and the value returned by **strcmp( )** is zero. In the second call, the first character of "Jerry" doesn't match with the first character of "Ferry" and the result is 4, which is the numeric

difference between ASCII value of 'J' and ASCII value of 'F'. In the third call to **strcmp( )** "Jerry" doesn't match with "Jerry boy", because the null character at the end of "Jerry" doesn't match the blank in "Jerry boy". The value returned is -32, which is the value of null character minus the ASCII value of space, i.e., '\0' minus ' ', which is equal to -32.

The exact value of mismatch will rarely concern us. All we usually want to know is whether or not the first string is alphabetically before the second string. If it is, a negative value is returned; if it isn't, a positive value is returned. Any non-zero value means there is a mismatch. Try to implement this procedure into a function **xstrcmp( )**.

Two-Dimensional Array of Characters

In the last chapter we saw several examples of 2-dimensional integer arrays. Let's now look at a similar entity, but one dealing with characters. Our example program asks you to type your name. When you do so, it checks your name against a master list to see if you are worthy of entry to the palace. Here's the program...

```
#define FOUND 1
#define NOTFOUND 0
main( )
{
    char  masterlist[6][10] = {
                                "akshay",
                                "parag",
                                "raman",
                                "srinivas",
                                "gopal",
                                "rajesh"
                              } ;
    int  i, flag, a ;
    char  yourname[10] ;
```

```
    printf ( "\nEnter your name " ) ;
    scanf ( "%s", yourname ) ;

    flag = NOTFOUND ;
    for ( i = 0 ; i <= 5 ; i++ )
    {
        a = strcmp ( &masterlist[i][0], yourname ) ;
        if ( a == 0 )
        {
            printf ( "Welcome, you can enter the palace" ) ;
            flag = FOUND ;
            break ;
        }
    }

    if ( flag == NOTFOUND )
        printf ( "Sorry, you are a trespasser" ) ;
}
```

And here is the output for two sample runs of this program...

```
Enter your name dinesh
Sorry, you are a trespasser
Enter your name raman
Welcome, you can enter the palace
```

Notice how the two-dimensional character array has been initialized. The order of the subscripts in the array declaration is important. The first subscript gives the number of names in the array, while the second subscript gives the length of each item in the array.

Instead of initializing names, had these names been supplied from the keyboard, the program segment would have looked like this...

```
for ( i = 0 ; i <= 5 ; i++ )
    scanf ( "%s", &masterlist[i][0] ) ;
```

While comparing the strings through **strcmp( )**, note that the addresses of the strings are being passed to **strcmp( )**. As seen in the last section, if the two strings match, **strcmp( )** would return a value 0, otherwise it would return a non-zero value.

The variable **flag** is used to keep a record of whether the control did reach inside the **if** or not. To begin with, we set **flag** to NOTFOUND. Later through the loop if the names match, **flag** is set to FOUND. When the control reaches beyond the **for** loop, if **flag** is still set to NOTFOUND, it means none of the names in the **masterlist[ ][ ]** matched with the one supplied from the keyboard.

The names would be stored in the memory as shown in Figure 9.3. Note that each string ends with a '\0'. The arrangement as you can appreciate is similar to that of a two-dimensional numeric array.

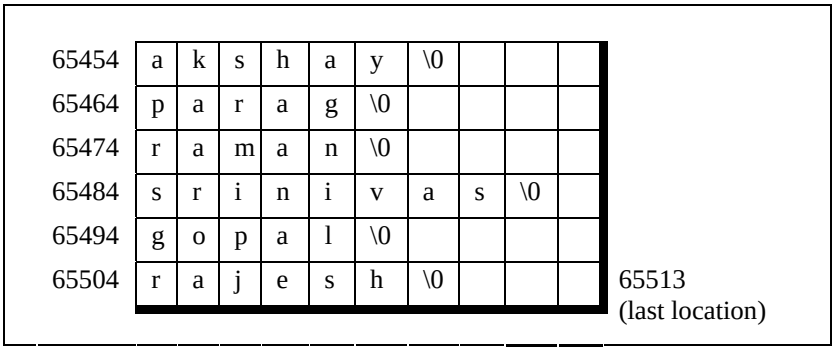

Figure 9.3

Here, 65454, 65464, 65474, etc. are the base addresses of successive names. As seen from the above pattern some of the names do not occupy all the bytes reserved for them. For example, even though 10 bytes are reserved for storing the name “akshay”, it occupies only 7 bytes. Thus, 3 bytes go waste. Similarly, for each name there is some amount of wastage. In fact, more the number of names, more would be the wastage. Can this not be avoided? Yes, it can be... by using what is called an ‘array of pointers’, which is our next topic of discussion.

Array of Pointers to Strings

As we know, a pointer variable always contains an address. Therefore, if we construct an array of pointers it would contain a number of addresses. Let us see how the names in the earlier example can be stored in the array of pointers.

```
char  *names[ ] = {
                        "akshay",
                        "parag",
                        "raman",
                        "srinivas",
                        "gopal",
                        "rajesh"
                  } ;
```

In this declaration **names[ ]** is an array of pointers. It contains base addresses of respective names. That is, base address of “akshay” is stored in **names[0]**, base address of “parag” is stored in **names[1]** and so on. This is depicted in Figure 9.4.

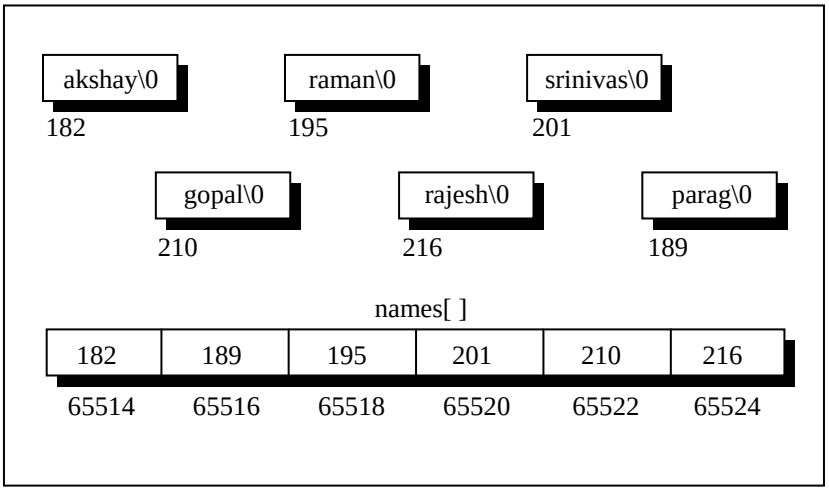

Figure 9.4

In the two-dimensional array of characters, the strings occupied 60 bytes. As against this, in array of pointers, the strings occupy only 41 bytes—a net saving of 19 bytes. A substantial saving, you would agree. But realize that actually 19 bytes are not saved, since 12 bytes are sacrificed for storing the addresses in the array **names[ ]**. Thus, one reason to store strings in an array of pointers is to make a more efficient use of available memory.

Another reason to use an array of pointers to store strings is to obtain greater ease in manipulation of the strings. This is shown by the following programs. The first one uses a two-dimensional array of characters to store the names, whereas the second uses an array of pointers to strings. The purpose of both the programs is very simple. We want to exchange the position of the names “raman” and “srinivas”.

```
/* Exchange names using 2-D array of characters */
main( )
{
      char  names[ ][10] = {
```

```
                                "akshay",
                                "parag",
                                "raman",
                                "srinivas",
                                "gopal",
                                "rajesh"
                              } ;
    int  i ;
    char  t ;

    printf ( "\nOriginal: %s %s", &names[2][0], &names[3][0] ) ;

    for ( i = 0 ; i <= 9 ; i++ )
    {
        t = names[2][i] ;
        names[2][i] = names[3][i] ;
        names[3][i] = t ;
    }

    printf ( "\nNew: %s %s", &names[2][0], &names[3][0] ) ;
}
```

And here is the output...

```
Original: raman srinivas
New: srinivas raman
```

Note that in this program to exchange the names we are required to exchange corresponding characters of the two names. In effect, 10 exchanges are needed to interchange two names.

Let us see, if the number of exchanges can be reduced by using an array of pointers to strings. Here is the program...

```
main( )
{
    char  *names[ ] = {
```

```
                                "akshay",
                                "parag",
                                "raman",
                                "srinivas",
                                "gopal",
                                "rajesh"
                            } ;
    char  *temp ;

    printf ( "Original: %s %s", names[2], names[3] ) ;

    temp = names[2] ;
    names[2] = names[3] ;
    names[3] = temp ;

    printf ( "\nNew: %s %s", names[2], names[3] ) ;
}
```

And here is the output...

```
Original: raman srinivas
New: srinivas raman
```

The output is same as the earlier program. In this program all that we are required to do is exchange the addresses (of the names) stored in the array of pointers, rather than the names themselves. Thus, by effecting just one exchange we are able to interchange names. This makes handling strings very convenient.

Thus, from the point of view of efficient memory usage and ease of programming, an array of pointers to strings definitely scores over a two-dimensional character array. That is why, even though in principle strings can be stored and handled through a two-dimensional array of characters, in actual practice it is the array of pointers to strings, which is more commonly used.

Limitation of Array of Pointers to Strings

When we are using a two-dimensional array of characters we are at liberty to either initialize the strings where we are declaring the array, or receive the strings using **scanf( )** function. However, when we are using an array of pointers to strings we can initialize the strings at the place where we are declaring the array, but we cannot receive the strings from keyboard using **scanf( )**. Thus, the following program would never work out.

```
main( )
{
    char  *names[6] ;
    int  i ;

    for ( i = 0 ; i <= 5 ; i++ )
    {
        printf ( "\nEnter name " ) ;
        scanf ( "%s", names[i] ) ;
    }
}
```

The program doesn't work because; when we are declaring the array it is containing garbage values. And it would be definitely wrong to send these garbage values to **scanf( )** as the addresses where it should keep the strings received from the keyboard.

Solution

If we are bent upon receiving the strings from keyboard using **scanf( )** and then storing their addresses in an array of pointers to strings we can do it in a slightly round about manner as shown below.

```
#include "alloc.h"
main( )
```

```
{
    char  *names[6] ;
    char n[50] ;
    int  len, i ;
    char *p ;

    for ( i = 0 ; i <= 5 ; i++ )
    {
        printf ( "\nEnter name " ) ;
        scanf ( "%s", n ) ;
        len = strlen ( n ) ;
        p = malloc ( len + 1 ) ;
        strcpy ( p, n ) ;
        names[i] = p ;
    }

    for ( i = 0 ; i <= 5 ; i++ )
        printf ( "\n%s", names[i] ) ;
}
```

Here we have first received a name using **scanf( )** in a string **n[ ]**. Then we have found out its length using **strlen( )** and allocated space for making a copy of this name. This memory allocation has been done using a standard library function called **malloc( )**. This function requires the number of bytes to be allocated and returns the base address of the chunk of memory that it allocates. The address returned by this function is always of the type **void ***. Hence it has been converted into **char *** using a feature called typecasting. Typecasting is discussed in detail in Chapter 15. The prototype of this function has been declared in the file 'alloc.h'. Hence we have **#includ**ed this file.

But why did we not use array to allocate memory? This is because with arrays we have to commit to the size of the array at the time of writing the program. Moreover, there is no way to increase or decrease the array size during execution of the program. In other words, when we use arrays static memory allocation takes place.

Unlike this, using **malloc( )** we can allocate memory dynamically, during execution. The argument that we pass to **malloc( )** can be a variable whose value can change during execution.

Once we have allocated the memory using **malloc( )** we have copied the name received through the keyboard into this allocated space and finally stored the address of the allocated chunk in the appropriate element of **names[ ]**, the array of pointers to strings.

This solution suffers in performance because we need to allocate memory and then do the copying of string for each name received through the keyboard.

Summary

(a) A string is nothing but an array of characters terminated by '\0'.
(b) Being an array, all the characters of a string are stored in contiguous memory locations.
(c) Though **scanf( )** can be used to receive multi-word strings, **gets( )** can do the same job in a cleaner way.
(d) Both **printf( )** and **puts( )** can handle multi-word strings.
(e) Strings can be operated upon using several standard library functions like **strlen( )**, **strcpy( )**, **strcat( )** and **strcmp( )** which can manipulate strings. More importantly we imitated some of these functions to learn how these standard library functions are written.
(f) Though in principle a 2-D array can be used to handle several strings, in practice an array of pointers to strings is preferred since it takes less space and is efficient in processing strings.
(g) **malloc( )** function can be used to allocate space in memory on the fly during execution of the program.

Exercise

Simple strings

[A] What would be the output of the following programs:

(a)
```
main( )
{
    char  c[2] = "A" ;
    printf ( "\n%c", c[0] ) ;
    printf ( "\n%s", c ) ;
}
```

(b)
```
main( )
{
    char  s[ ] = "Get organised! learn C!!" ;
    printf ( "\n%s", &s[2] ) ;
    printf ( "\n%s", s ) ;
    printf ( "\n%s", &s ) ;
    printf ( "\n%c", s[2] ) ;
}
```

(c)
```
main( )
{
    char  s[ ] = "No two viruses work similarly" ;
    int  i = 0 ;
    while ( s[i] != 0 )
    {
        printf ( "\n%c %c", s[i], *( s + i ) ) ;
        printf ( "\n%c %c", i[s], *( i + s ) ) ;
        i++ ;
    }
}
```

(d)
```
main( )
{
    char  s[ ] = "Churchgate: no church no gate" ;
    char  t[25] ;
    char  *ss, *tt ;
    ss = s ;
    while ( *ss != '\0' )
        *ss++ = *tt++ ;
```

```
        printf ( "\n%s", t ) ;
    }
```

(e)
```
main( )
{
    char str1[ ] = { 'H', 'e', 'l', 'l', 'o'  } ;
    char str2[ ] = "Hello" ;

    printf ( "\n%s", str1 ) ;
    printf ( "\n%s", str2 ) ;
}
```

(f)
```
main(  )
{
    printf ( 5 + "Good Morning " ) ;
}
```

(g)
```
main( )
{
    printf ( "%c",  "abcdefgh"[4] ) ;
}
```

(h)
```
main( )
{
    printf ( "\n%d%d", sizeof ( '3' ), sizeof ( "3" ), sizeof ( 3 ) ) ;
}
```

[B] Point out the errors, if any, in the following programs:

(a)
```
main( )
{
    char *str1 = "United" ;
    char *str2 = "Front" ;
    char *str3 ;
    str3 = strcat ( str1, str2 ) ;
    printf ( "\n%s", str3 ) ;
}
```

(b)
```
main( )
{
```

```
    int  arr[ ] = { 'A', 'B', 'C', 'D' } ;
    int  i ;
    for ( i = 0 ; i <= 3 ; i++ )
        printf ( "\n%d", arr[i] ) ;
}
```

(c)
```
main( )
{
    char  arr[8] = "Rhombus" ;
    int  i ;
    for ( i = 0 ; i <= 7 ; i++ )
        printf ( "\n%d", *arr ) ;
        arr++ ;
}
```

[C] Fill in the blanks:

(a) "A" is a _____ while 'A' is a _____.

(b) A string is terminated by a _____ character, which is written as _____.

(c) The array **char name[10]** can consist of a maximum of _____ characters.

(d) The array elements are always stored in _____ memory locations.

[D] Attempt the following:

(a) Which is more appropriate for reading in a multi-word string?

gets( ) printf( ) scanf( ) puts( )

(b) If the string "Alice in wonder land" is fed to the following **scanf( )** statement, what will be the contents of the arrays **str1**, **str2**, **str3** and **str4**?

```
scanf ( "%s%s%s%s%s", str1, str2, str3, str4 ) ;
```

(c) Write a program that converts all lowercase characters in a given string to its equivalent uppercase character.

(d) Write a program that extracts part of the given string from the specified position. For example, if the sting is "Working with strings is fun", then if from position 4, 4 characters are to be extracted then the program should return string as "king". Moreover, if the position from where the string is to be extracted is given and the number of characters to be extracted is 0 then the program should extract entire string from the specified position.

(e) Write a program that converts a string like "124" to an integer 124.

(f) Write a program that replaces two or more consecutive blanks in a string by a single blank. For example, if the input is

Grim return to the planet of apes!!

the output should be

Grim return to the planet of apes!!

Two-dimensional array, Array of pointers to strings

[E] Answer the following:

(a) How many bytes in memory would be occupied by the following array of pointers to strings? How many bytes would be required to store the same strings, if they are stored in a two-dimensional character array?

```
char  *mess[ ] =  {
                        "Hammer and tongs",
                        "Tooth and nail",
```

```
        "Spit and polish",
        "You and C"
    } ;
```

(b) Can an array of pointers to strings be used to collect strings from the keyboard? If not, why not?

[F] Attempt the following:

(a) Write a program that uses an array of pointers to strings **str**[ ]. Receive two strings **str1** and **str2** and check if **str1** is embedded in any of the strings in **str[ ]**. If str1 is found, then replace it with **str2**.

```
char *str[ ] = {
        "We will teach you how to...",
        "Move a mountain",
        "Level a building",
        "Erase the past",
        "Make a million",
        "...all through C!"
    } ;
```

For example if **str1** contains "mountain" and **str2** contains "car", then the second string in **str** should get changed to "Move a car".

(b) Write a program to sort a set of names stored in an array in alphabetical order.

(c) Write a program to reverse the strings stored in the following array of pointers to strings:

```
char  *s[ ] = {
        "To err is human...",
        "But to really mess things up...",
        "One needs to know C!!"
    } ;
```

Hint: Write a function **xstrrev (string)** which should reverse the contents of one string. Call this function for reversing each string stored in **s**.

(d) Develop a program that receives the month and year from the keyboard as integers and prints the calendar in the following format.

September 2004

Mon	Tue	Wed	Thu	Fri	Sat	Sun
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

Note that according to the Gregorian calendar 01/01/1900 was Monday. With this as the base the calendar should be generated.

(e) Modify the above program suitably so that once the calendar for a particular month and year has been displayed on the

screen, then using arrow keys the user must be able to change the calendar in the following manner:

Up arrow key	: Next year, same month
Down arrow key	: Previous year, same month
Right arrow key	: Same year, next month
Left arrow key	: Same year, previous month

If the escape key is hit then the procedure should stop.

Hint: Use the **getkey( )** function discussed in Chapter 8, problem number [L](c).

(f) A factory has 3 division and stocks 4 categories of products. An inventory table is updated for each division and for each product as they are received. There are three independent suppliers of products to the factory:

(a) Design a data format to represent each transaction.
(b) Write a program to take a transaction and update the inventory.
(c) If the cost per item is also given write a program to calculate the total inventory values.

(g) A dequeue is an ordered set of elements in which elements may be inserted or retrieved from either end. Using an array simulate a dequeue of characters and the operations retrieve left, retrieve right, insert left, insert right. Exceptional conditions such as dequeue full or empty should be indicated. Two pointers (namely, left and right) are needed in this simulation.

(h) Write a program to delete all vowels from a sentence. Assume that the sentence is not more than 80 characters long.

(i) Write a program that will read a line and delete from it all occurrences of the word ‘the’.

(j) Write a program that takes a set of names of individuals and abbreviates the first, middle and other names except the last name by their first letter.

(k) Write a program to count the number of occurrences of any two vowels in succession in a line of text. For example, in the sentence

"Pleases read this application and give me gratuity"

such occurrences are ea, ea, ui.